Portfolio 1 — Reflection Report

Per Rune Overelv
Frontend Development — Noroff
Portfolio 1

All links

Portfolio website (live): https://comfy-stardust-d1542b.netlify.app/

Personal GitHub profile: https://github.com/Runeov

Project	GitHub repository	Live website
Contagious Coffee (Cross Course Project)	github.com/Runeov/HTML-CSS-Course-Assignment	magical-genie-a5c987.netlify.app
Hiking Project (Semester Project 1)	github.com/Runeov/Hiking-project-new	guileless-salmiakki-0c085e.netlify.app
NorgeTravel	github.com/Runeov/norgetravel	norgetravel.com

Introduction

This portfolio is a self-assessment of three projects I built during my first year in the Noroff Frontend Development programme. For each project I describe what I built, what I learned, and — most importantly — the concrete improvements I would make now that I have more skills. Looking back at older work with fresh eyes is the clearest measure of how much I have grown, and writing down these improvements helps me keep raising my own standard as a developer.

1. Contagious Coffee — Cross Course Project

What it is: A coffee-shop website for Contagious Coffee that helps customers find their nearest café, hand-coded with semantic HTML and CSS.

Reflection: This was where I learned to translate a visual design into structured, semantic markup and to keep my CSS organised so styles stayed predictable across pages. It taught me the value of planning the HTML structure before writing any styling.

Things I would improve:

- **Accessibility:** add descriptive `alt` text to every image, check colour contrast against WCAG guidelines, and add clear keyboard focus states so the site is usable without a mouse.
- **Responsiveness:** rebuild the layout mobile-first with tested breakpoints instead of relying on fixed widths, so it looks good on phones, tablets, and desktops.
- **Cleaner CSS:** refactor repeated rules into reusable classes and CSS custom properties (variables) for colours and spacing, which would cut duplication and make future changes faster.
- **Performance:** compress and correctly size images, and add `width`/`height` attributes to prevent layout shift while the page loads.
- **Maintainability:** adopt a consistent, semantic naming convention (such as BEM) and a tidier file structure so the code is easier to read and extend.

2. Hiking Project — Semester Project 1

What it is: A multi-page site for hiking enthusiasts, built mobile-first with accessible layouts and reusable components.

Reflection: This project pushed me to plan a multi-page site, think about how the layout should behave across screen sizes, and reuse components instead of repeating code. Managing several pages taught me how important consistency and structure are as a project grows.

Things I would improve:

- **Responsive polish:** tighten the breakpoints so the layout holds up on very small phones and very wide desktops, and test on real devices rather than just the browser resizer.
- **Accessibility:** use a logical heading hierarchy, meaningful `alt` text, ARIA labels on interactive elements, and visible focus styles throughout.
- **Less duplication:** extract repeated sections such as the header, footer, and cards into reusable patterns to keep the markup DRY.
- **Performance:** compress images and remove unused CSS to reduce page weight and improve load times.
- **Interactivity:** add small, progressive JavaScript enhancements — for example a mobile navigation toggle and client-side form validation — to improve the user experience.

3. NorgeTravel

What it is: A travel platform guiding visitors to Norway's Northern Lights and Arctic adventures, built with Next.js, TypeScript, and Tailwind CSS.

Reflection: This project stretched me beyond plain HTML and CSS into a component-based framework, TypeScript, and SEO/structured data. It showed me how much more maintainable a larger site becomes when it is built from reusable components and typed code, and how much detail goes into making a site discoverable and fast.

Things I would improve:

- **Accessibility audit:** systematically check and fix colour contrast, `alt` text, keyboard navigation, and semantic landmarks across every page.
- **Performance:** run a Lighthouse pass and act on it — optimise images with `next/image`, lazy-load below-the-fold content, and reduce the JavaScript bundle size.
- **Testing and types:** add component/unit tests and tighten the TypeScript types so errors are caught before they reach production.
- **SEO depth:** complete the metadata on every route, keep the sitemap current, and validate the structured data with Google's Rich Results test.
- **Scalability:** refine the shared components so new destinations and tours can be added with minimal repeated work.

Conclusion

Comparing these three projects shows how far I have come: from writing my first semantic HTML and CSS, to planning multi-page layouts, to building a typed, component-based application with a real framework. The common thread in everything I would improve is the same — stronger accessibility, better performance, and cleaner, more reusable code. Those are the goals I am carrying into the rest of the programme as I continue to grow as a Frontend Developer.